

React Props & Hooks

Pass data between components with props, and master every core hook — `useState`, `useEffect`, `useRef`, `useReducer`, `useMemo`, `useCallback` — plus custom hooks and rendering performance.

WHAT WE COVER

[Props vs State](#)[Lifecycle](#)[All the hooks](#)[Custom hooks](#)[HOCs](#)[Performance](#)

On Day 10 you built components and used `useState`. Today we go deep. First, props — how a parent passes data down to its children, and how children talk back. Then the full set of React hooks, each with a clear example and when to use it. We'll cover the component lifecycle, refs, higher-order components, and how rendering affects performance — then build a small app that styles components from props and manages everything with hooks.

React Props & Hooks

Props make components **reusable**; hooks make them **powerful**. This is the core of day-to-day React: passing data down, reacting to changes, holding references, and keeping the UI fast. Everything is typed with the interfaces from Days 8-9.

CLASS AGENDA · ~2 HOURS

0:00	Props, props vs state & styling from props	§1-3
0:20	Events, child-to-parent & the lifecycle	§4-5
0:40	useState & useEffect in depth	§6-7
1:00	useRef, useReducer, useMemo, useCallback	§8-11
1:25	Custom hooks, performance & HOCs	§12-14
1:45	Build the project & wrap-up	§15

ON THIS HANDOUT

- | | |
|--|---|
| 01. Props: passing data down | 09. useReducer: structured state |
| 02. Props vs State | 10. useMemo: cache a calculation |
| 03. Styling components from props | 11. useCallback & React.memo |
| 04. Events & talking back to a parent | 12. Custom hooks |
| 05. The component lifecycle | 13. Rendering & performance |
| 06. useState in depth | 14. Higher-Order Components |
| 07. useEffect: side effects | 15. Project: Team Board |
| 08. useRef: references & values | |

PART 1 · Props & the Component Model

01 Props: passing data down

Props (properties) are how a parent component passes data **down** to a child. You write them like HTML attributes; the child receives them as an object. Type them with an interface (Day 9) so every use is checked.

```

interface CardProps {
  name: string;
  role: string;
}

function Card({ name, role }: CardProps) {  // destructure the props
  return (
    <div className="card">
      <h3>{name}</h3>
      <p>{role}</p>
    </div>
  );
}

function App() {
  return (
    <div>
      <Card name="Abhishek" role="Admin" />  {/* pass props like attributes */}
      <Card name="Sam" role="Editor" />      {/* reuse with different data */}
    </div>
  );
}

```

TSX

- ▶ The parent sets props; the child reads them. This is **one-way data flow** (top to bottom).
- ▶ Props are **read-only** inside the child — never modify them.
- ▶ Typing props with an interface means a missing or wrong prop is caught by the compiler.

02 Props vs State

This distinction trips everyone up, so make it crisp:

- ▶ **Props** come from the parent, are read-only, and describe how a component is configured.
- ▶ **State** is owned by the component, is changeable (via its setter), and describes what changes over time.
- ▶ Rule of thumb: if a value is passed in, it's a prop; if the component manages and changes it itself, it's state.

```

function Profile({ name }: { name: string }) {  // name is a PROP (from parent)
  const [likes, setLikes] = useState<number>(0);  // likes is STATE (owned here)

  return (
    <div>
      <h3>{name}</h3>
      <button onClick={() => setLikes(likes + 1)}>Likes: {likes}</button>
    </div>
  );
}

```

TSX

A component re-renders when either its props change (parent passes new ones) or its own state changes.

03 Styling components from props

A powerful pattern: a component decides its own look based on the props it receives. Just compute a class name (or style) from the prop.

```
interface BadgeProps {  
  label: string;  
  online: boolean;  
}  
  
function Badge({ label, online }: BadgeProps) {  
  const cls = online ? "badge badge--green" : "badge badge--grey";  
  return <span className={cls}>{label}</span>;  
}  
  
// or a variant prop that maps to different styles:  
function Button({ variant }: { variant: "primary" | "danger" }) {  
  return <button className={`btn btn--${variant}`}>Click</button>;  
}
```

TSX

- ▶ Compute the class from props, then let CSS (or Tailwind) do the visuals.
- ▶ A literal-union prop (`"primary" | "danger"`) keeps variants type-safe.
- ▶ This is exactly what you'll do in today's project.

04 Events & talking back to a parent

Data flows down through props; to send information **back up**, the parent passes a **function** as a prop, and the child calls it. This is how a child “talks to” its parent.

```

// child receives a callback prop
function SearchBar({ onSearch }: { onSearch: (q: string) => void }) {
  return (
    <input
      placeholder="Search..."
      onChange={e => onSearch(e.target.value)} // call the parent's function
    />
  );
}

function App() {
  const [query, setQuery] = useState<string>("");
  return (
    <div>
      <SearchBar onSearch={setQuery} /> /* pass a function down */
      <p>Searching for: {query}</p>
    </div>
  );
}

```

- ▶ Parent passes a function prop; the child calls it to report events or data.
- ▶ The event object is typed for you here (`e.target.value` is a string).

05 The component lifecycle

Every component goes through three phases. Understanding them explains when your code runs — and it's all controlled through `useEffect`.

- ▶ **Mounting** — the component is created and its JSX is added to the page (appears).
- ▶ **Updating** — its props or state change, so it re-renders.
- ▶ **Unmounting** — the component is removed from the page (disappears).

```

function Timer() {
  useEffect(() => {
    console.log("mounted"); // runs after first render (mount)
    const id = setInterval(() => console.log("tick"), 1000);

    return () => {
      clearInterval(id);
      console.log("unmounted"); // cleanup runs on unmount
    };
  }, []); // empty deps: set up once on mount, clean up on unmount

  return <p>Look at the console.</p>;
}

```

So JSX describes what to render; mounting adds it; state/prop changes update it; and when it leaves, cleanup runs. Next we'll see each hook that plugs into this.

PART 2 · The Hooks, In Depth

06 useState in depth

`useState` holds data that changes over time and re-renders the component when it does. A few patterns beyond the basics:

```
const [count, setCount] = useState<number>(0);

// functional update -- use when the new value depends on the old one
setCount((prev) => prev + 1);

// state can be an object or array (always create a NEW one, don't mutate)
const [user, setUser] = useState({ name: "Abhi", age: 25 });
setUser((prev) => ({ ...prev, age: prev.age + 1 })); // copy, then change

const [items, setItems] = useState<string[]>([]);
setItems((prev) => [...prev, "new item"]); // add without mutating
```

- ▶ Use the **functional update** form when the next value depends on the previous one.
- ▶ Never mutate state directly — always pass a new object/array so React notices the change.

07 useEffect: side effects

`useEffect` runs **side effects** — things outside rendering: fetching data, timers, subscriptions, manual DOM work. The dependency array controls when it runs.

```
// 1) run once, on mount (e.g. fetch data)
useEffect(() => {
  fetch("/api/tasks").then((r) => r.json()).then(setTasks);
}, []);

// 2) run whenever a value changes
useEffect(() => {
  document.title = `Count: ${count}`;
}, [count]); // re-runs each time count changes

// 3) clean up (timers, listeners)
useEffect(() => {
  const id = setInterval(tick, 1000);
  return () => clearInterval(id); // cleanup before next run / on unmount
}, []);
```

- ▶ `[]` = run once on mount. `[a, b]` = run when a or b changes. No array = run every render (rarely what you want).
- ▶ Return a function to clean up (clear timers, remove listeners) — prevents leaks.

- ▶ This is where you'll fetch data (reusing your Day 5 async + Day 7 fetch knowledge).

08 useRef: references & values

`useRef` does two jobs: it gives you a handle to a real DOM element, and it stores a mutable value that **survives** re-renders without causing one.

```
// 1) reference a DOM element
function NameField() {
  const inputRef = useRef<HTMLInputElement>(null);
  return (
    <div>
      <input ref={inputRef} />
      <button onClick={() => inputRef.current?.focus()}>Focus</button>
    </div>
  );
}

// 2) a value that persists but does NOT trigger re-renders
const renderCount = useRef<number>(0);
renderCount.current++; // changing .current does not re-render
```

TSX

- ▶ Access the element or value through `.current`.
- ▶ Unlike state, changing a ref does **not** re-render — use it for values the UI doesn't display.

09 useReducer: structured state

When state has several related pieces or complex update logic, `useReducer` is cleaner than many `useState` s. You **dispatch actions**, and a **reducer** function decides the next state — exactly the switch pattern from Day 4.

```

type State = { count: number };
type Action = { type: "inc" } | { type: "dec" } | { type: "reset" };

function reducer(state: State, action: Action): State {
  switch (action.type) {
    case "inc": return { count: state.count + 1 };
    case "dec": return { count: state.count - 1 };
    case "reset": return { count: 0 };
  }
}

function Counter() {
  const [state, dispatch] = useReducer(reducer, { count: 0 });
  return (
    <div>
      <span>{state.count}</span>
      <button onClick={() => dispatch({ type: "inc" })}>+</button>
      <button onClick={() => dispatch({ type: "dec" })}>-</button>
    </div>
  );
}

```

TSX

- ▶ All update logic lives in one predictable place (the reducer).
- ▶ Great for forms, multi-step flows, or anything with many related updates.

10 useMemo: cache a calculation

`useMemo` remembers the result of an expensive calculation and only recomputes it when its dependencies change — avoiding wasted work on every render.

```

function ProductList({ products, query }: { products: Product[]; query: string }) {
  // only re-filters when products or query change, not on every render
  const filtered = useMemo(
    () => products.filter((p) => p.name.includes(query)),
    [products, query]
  );

  return <ul>{filtered.map((p) => <li key={p.id}>{p.name}</li>)}</ul>;
}

```

TSX

- ▶ Wrap a costly computation; pass the values it depends on in the array.
- ▶ Don't overuse it — reach for it when a calculation is genuinely heavy or a list is large.

11 useCallback & React.memo

`useCallback` remembers a **function** so it isn't recreated every render. Paired with `React.memo` (which skips re-rendering a child whose props haven't changed), it prevents needless child re-renders.


```

const Child = memo(function Child({ onClick }: { onClick: () => void }) {
  console.log("child rendered");
  return <button onClick={onClick}>Click</button>;
});

function Parent() {
  const [count, setCount] = useState(0);

  // same function identity across renders -> Child won't re-render needlessly
  const handleClick = useCallback(() => console.log("clicked"), []);

  return (
    <div>
      <button onClick={() => setCount(count + 1)}>Count {count}</button>
      <Child onClick={handleClick} />
    </div>
  );
}

```

- ▶ `memo` skips re-rendering a child when its props are unchanged.
- ▶ `useCallback` keeps a function's identity stable so `memo` can do its job.

12 Custom hooks

A **custom hook** is just a function whose name starts with `use` and that calls other hooks. It lets you **extract and reuse** stateful logic across components — the cleanest way to stay DRY in React.

```

// reusable toggle logic
function useToggle(initial: boolean): [boolean, () => void] {
  const [on, setOn] = useState<boolean>(initial);
  const toggle = useCallback(() => setOn((v) => !v), []);
  return [on, toggle];
}

// now any component can reuse it:
function Panel() {
  const [open, toggleOpen] = useToggle(false);
  return (
    <div>
      <button onClick={toggleOpen}>{open ? "Hide" : "Show"}</button>
      {open && <p>Now you see me.</p>}
    </div>
  );
}

```

- ▶ Custom hooks share **logic**, not UI — perfect for things like `useToggle`, `useLocalStorage`, `useFetch`.
- ▶ They follow the same rules as built-in hooks (call them at the top level of a component or another hook).

TRY IT YOURSELF

Write a `useCounter(start)` hook returning `{ count, increment, reset }` and use it in two different components.

PART 3 · Performance & Patterns

13 Rendering & performance

Understanding **when** React re-renders helps you keep apps fast.

- ▶ A component re-renders when its **state** changes, or when its **parent** re-renders (passing possibly-new props).
- ▶ Re-rendering is usually cheap — don't optimise blindly. Measure first.
- ▶ When a list or subtree is genuinely expensive, reach for `React.memo`, `useMemo`, and `useCallback`.
- ▶ Always give list items a stable `key` — it lets React update only what changed instead of re-building the list.
- ▶ Avoid creating brand-new objects/functions inline as props to memoised children (that defeats the memo).

The short version: write clear code first; add memoisation only where a real bottleneck shows up.

14 Higher-Order Components

A **Higher-Order Component (HOC)** is a function that takes a component and returns a new, enhanced one. It's an older pattern for sharing behaviour; custom hooks now cover most cases, but you'll still see HOCs in real codebases.

```
// adds a loading state to any component
function withLoading<P extends object>(Component: React.ComponentType<P>) {
  return function Wrapped(props: P & { loading: boolean }) {
    if (props.loading) return <p>Loading...</p>;
    return <Component {...(props as P)} />;
  };
}

const UserCardWithLoading = withLoading(UserCard);
// <UserCardWithLoading name="Abhi" loading={true} /> -> shows "Loading..."
```

TSX

- ▶ An HOC wraps a component to add shared behaviour (loading, auth checks, logging).
- ▶ Modern tip: prefer a **custom hook** for logic; use HOCs mainly when wrapping the rendered output.

PART 4 • Project

15 Project: Team Board

Build a small “Team Board” that shows member cards, styled by their status, with a live search. It exercises props, prop-based styling, and several hooks together.

What it does

- ▶ Shows a grid of member cards loaded on mount.
- ▶ Each card styles itself from its props (online = green accent, offline = grey).
- ▶ A search box (auto-focused with a ref) filters the list as you type.

The card (styled from props)

```
interface Member { id: number; name: string; role: string; online: boolean; }  
  
function MemberCard({ name, role, online }: Omit<Member, "id">) {  
  const cls = online ? "card card--online" : "card card--offline";  
  return (  
    <div className={cls}>  
      <h3>{name}</h3>  
      <p>{role}</p>  
      <span>{online ? "Online" : "Offline"}</span>  
    </div>  
  );  
}
```

TSX

The board (hooks tie it together)

```
function TeamBoard() {
  const [members, setMembers] = useState<Member[]>([]);
  const [query, setQuery] = useState<string>("");
  const searchRef = useRef<HTMLInputElement>(null);

  useEffect(() => { // load once on mount
    setMembers([
      { id: 1, name: "Abhishek", role: "Admin", online: true },
      { id: 2, name: "Sam", role: "Editor", online: false },
      { id: 3, name: "Riya", role: "Design", online: true },
    ]);
    searchRef.current?.focus(); // focus the search box
  }, []);

  const visible = members.filter((m) =>
    m.name.toLowerCase().includes(query.toLowerCase())
  );

  return (
    <div>
      <input ref={searchRef} value={query}
        onChange={(e) => setQuery(e.target.value)}
        placeholder="Search members..." />
      <div className="grid">
        {visible.map((m) => (
          <MemberCard key={m.id} name={m.name} role={m.role} online={m.online} />
        ))}
      </div>
    </div>
  );
}
```

- ▶ Props + prop-based styling: `MemberCard` changes its look from its `online` prop.
- ▶ Hooks: `useState` for data and query, `useEffect` to load + focus, `useRef` for the input.
- ▶ Stretch: extract the search logic into a `useSearch` custom hook, or add a `useToggle` for a compact view.

Practice Challenges

Reinforce today with a few of these:

- ▶ Add an online/offline **toggle** button to each card that updates that member's state.
- ▶ Write a `useLocalStorage` custom hook and use it to remember the search text.
- ▶ Show a live count ("3 of 5 online") computed with `useMemo`.
- ▶ Convert the counter to `useReducer` with inc / dec / reset actions.
- ▶ Wrap `MemberCard` in `React.memo` and confirm it only re-renders when its props change.

Conclusion & what's next

That's the heart of React. You can pass data down with **props**, distinguish props from **state**, style components from their props, send data back up with callback props, and use every core **hook** — including custom hooks — while keeping rendering efficient.

Next class (Day 12): React Router — turning your single-page app into a multi-page experience with real URLs, nested layouts, and dynamic routes.

QUICK RECAP

Props flow data down (read-only); **state** is owned & changeable · children talk back via **callback props** · the lifecycle (mount / update / unmount) runs through **useEffect** · master the hooks: **useState**, **useEffect**, **useRef**, **useReducer**, **useMemo**, **useCallback** & custom hooks · memoise only real bottlenecks.